

Dossier Final - Projet Mbeund-Mi (Thiaroye Sur Mer)

Plateforme de prévention et d'alerte des inondations

Ce document présente l'architecture globale du projet, détaille étape par étape le travail de chaque membre de l'équipe, et décrit les mesures de sécurité critiques à respecter avant toute publication (GitHub).

1. Travail de l'équipe : Étape par Étape

A. Maïmouna Sall (Coordinatrice IA & API)

Mission : Cerveau du système, elle relie les données physiques, l'intelligence artificielle et le backend.

- **Étape 1 : Collecte de Données (IoT & Météo).** Mise en place du script `fetch_historical_weather.py` pour récupérer l'historique météo de Thiaroye (2010-2024) via Open-Meteo, et du simulateur de capteurs IoT (MQTT) pour reproduire la montée des eaux en direct.
- **Étape 2 : Intelligence Artificielle (Machine Learning).** Entraînement d'un réseau de neurones LSTM sur des fenêtres temporelles de 24h pour prédire la montée de l'eau, couplé à un **Random Forest** pour classifier le niveau de risque (Vert, Jaune, Orange, Rouge).
- **Étape 3 : Alertes Multicanales.** Configuration de Twilio (`declencheur_alertes.py`) pour envoyer des SMS d'urgence automatisés, avec un système "Anti-Spam" pour ne pas saturer les téléphones des autorités.
- **Étape 4 : Télédétection.** Utilisation de **Google Earth Engine (GEE)** pour analyser les images satellites (Sentinel-2) et cartographier l'eau (indice NDWI) sur l'ensemble du périmètre de Thiaroye.

B. Mama Adam & Mame Diarra (Backend, Base de Données & Infrastructure)

Mission : Cœur du système, elles assurent le stockage des données, l'hébergement de l'API et la logique métier.

- **Étape 1 : Conception de l'Architecture.** Création d'un projet Django isolé (`mbeund_mi_backend`) pour garantir la robustesse.
- **Étape 2 : Modélisation des Données (SQLite / PostgreSQL).** Création des tables relationnelles (Zone, Mesure, Alerte, Signalement) pour structurer toutes les informations qui transitent.
- **Étape 3 : Création de l'API REST.** Développement des "Endpoints" via Django REST Framework (`/api/alertes/`, etc.) qui permettent aux autres services de lire et d'écrire dans la base de données.
- **Étape 4 : Gestion des Signalements & Geofencing.** Implémentation du système de signalements citoyens avec une sécurité géographique stricte : la formule de Haversine bloque automatiquement les requêtes provenant de personnes situées à plus de 4 km de

Thiaroye.

- **Étape 5 : Compression Serveur.** Mise en place de la librairie **Pillow** pour compresser instantanément (à 100 Ko) les photos envoyées par les citoyens, afin de préserver l'espace disque du serveur.

C. Ngoné Gueye (Frontend / Application Mobile & Web)

Mission : L'interface utilisateur, elle rend le système accessible et compréhensible pour la population et les autorités.

- **Étape 1 : Connexion au Backend.** Utilisation de requêtes HTTP (via Axios ou Fetch) pour consommer l'API REST créée par Mama Adam et récupérer les alertes et les zones en temps réel.
 - **Étape 2 : Cartographie.** Affichage de la carte de Thiaroye Sur Mer (Leaflet/Mapbox) en plaçant les points de capteurs et en colorisant les zones selon le risque (Vert à Rouge).
 - **Étape 3 : Notifications Push (Firebase).** Intégration de Firebase Cloud Messaging. Grâce à la synchronisation avec le Backend, l'application reçoit et affiche des pop-ups d'alerte en une fraction de seconde.
 - **Étape 4 : Interface de Crowdsourcing.** Création du formulaire de signalement citoyen permettant aux habitants d'envoyer des photos et de catégoriser le problème (Égouts, Inondation).
-

2. Mesures de Sécurité Critiques (Avant publication sur GitHub)

Si ce projet est publié sur GitHub sans précaution, des pirates informatiques scanneront le code en quelques secondes pour voler vos clés d'API, ce qui pourrait vous coûter des milliers de dollars (Twilio) ou compromettre les données utilisateurs (Firebase).

Voici la check-list de sécurité (déjà appliquée sur votre projet) :

1. Fichier `.env` (Environnement Virtuel)

Jamais de mots de passe "en dur" dans le code. Toutes les clés sensibles ont été transférées dans un fichier caché nommé `.env` à la racine de votre dossier `backend` :

- `SECRET_KEY` (Clé de chiffrement de Django)
- `TWILIO_ACCOUNT_SID` et `TWILIO_AUTH_TOKEN` (Vos accès SMS)
- Le chemin vers vos identifiants Firebase.
- *Le code Python utilise désormais la librairie `python-dotenv` pour lire ces informations de manière sécurisée.*

2. Le bouclier `.gitignore`

Interdire à Git d'exporter les secrets. Un fichier `.gitignore` a été créé à la racine du projet. Il indique très clairement à Git d'ignorer et de **ne jamais uploader** les fichiers suivants :

- `.env` (Vos mots de passe)
- `firebase_credentials.json` (Le passe-partout de votre compte Google)

- `db.sqlite3` (Toute la base de données locale)
- `env/` (L'environnement virtuel lourd)

3. Les règles pour le Jour J (Déploiement en Production)

Le jour où Mama Adam déploiera le Backend sur un vrai serveur sur internet (ex: Render, Heroku) :

- Il faudra ouvrir `backend/mbeund_mi_backend/settings.py` et changer la ligne `DEBUG = True` en **`DEBUG = False`**. Cela empêchera les pirates de voir les détails techniques du code s'il y a une erreur.
- Il faudra manuellement ajouter le contenu du fichier `.env` dans les "Environment Variables" du tableau de bord de votre hébergeur web.

Validation Finale : Le projet Mbeund-Mi respecte aujourd'hui les standards professionnels de sécurité et d'architecture. Vous êtes prêts !